MSBA 402 – Fall 2026

Intro to SQL and Basic Data Management

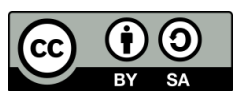
Lecture 2

October 5, 2026

Prof. Ryan Rosario

Anderson School of Management – Business Analytics

University of California, Los Angeles



This work is published under a Creative Commons BY-SA license.

Outline

- Introduction to Databases

- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually

- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

Database Purpose

A database *abstracts away* how the data is stored, maintained and processed. A database:

- Provides a way to create, read, update and delete data without worrying about files and breaking data integrity.
- Provides ONE single location for all data in the database (though the database itself may be distributed over multiple nodes, we don't care).

Schemas

More importantly, the overall *design* of a database is called a *schema*.

The term *schema* is sometimes used interchangeably to describe the logical structure of a **database** or a **relation** (table).

In other texts a *subschema* refers to the design of a particular part of the database, but outside of that text, we usually still call it a *schema*, if it refers to a DB or a table.

Data Model

Data models define how we design databases or interact with data. In particular:

- How do we define data?
- How we encode relationships among data?
- How we impose constraints on data?

In this class, we work with the **relational** model.

The Relational Model

Data is stored as a *relation* (implemented as a **table**). **Rows** represent individual transactions, records, or data points, called *tuples*. **Columns** represent *attributes* common to all rows and have specific *types* and appear in same order across rows.

name	**type**	**sun_requirement**	**height_in**
Rose	Perennial	Full Sun	48
Marigold	Annual	Full Sun	12
Tulip	Perennial	Full Sun	18
Impatiens	Annual	Partial Shade	12
Sunflower	Annual	Full Sun	96
Hosta	Perennial	Shade	24
Petunia	Annual	Full Sun	12
Daffodil	Perennial	Partial Shade	16
⋮	⋮	⋮	⋮

What is a Relation?

- A *relational database* consists of a collection of *relations* (tables). For the rest of this discussion we will use more conventional terms.
- Each table has zero or more rows. Each row is a transaction, a record, or a data point.
- Each row contains one or more columns. Each column (attribute) has a **type** with a *domain*, a set of legal values. This domain can be discrete or continuous.
- Each row contains the same columns in the same order.
- A column can contain `NULL` values, which simply means "unknown" or "missing".
- One or more columns form a unique identifier called a **key**.

"Database Languages"

There are many semantic systems for working with databases:

1. Data Definition Language (DDL) *(today)*
2. Data Query Language (DQL) *(most of the quarter)*
3. Data Manipulation Language (DML) *(later)*
4. Data Control Language (DCL)
5. Transaction Control Language (TCL)

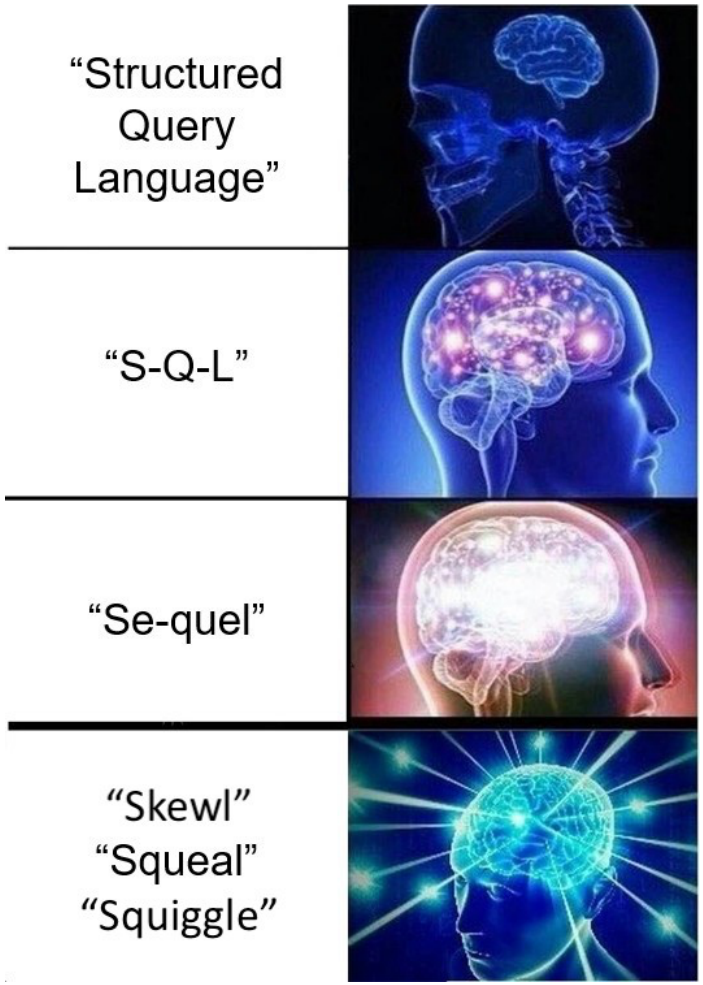
For the relational model, they all use SQL.

Structured Query Language

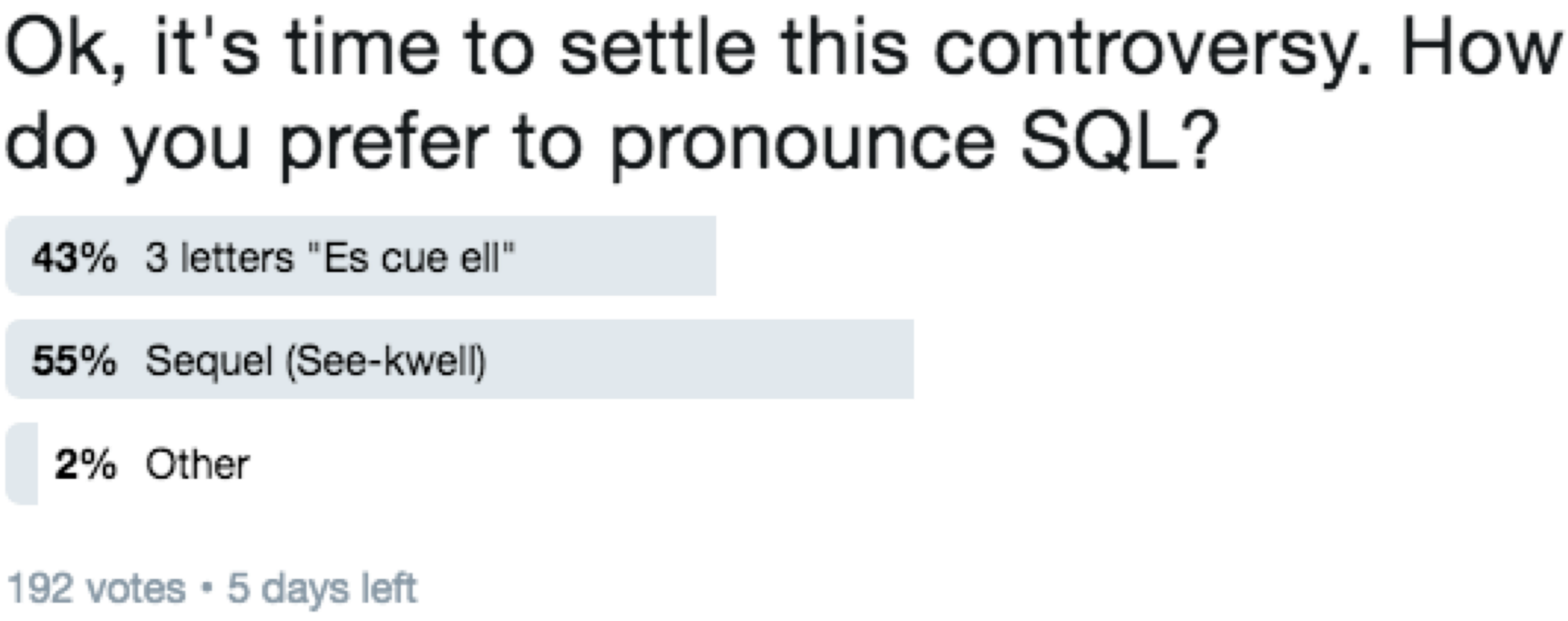
SQL is pronounced as either *Es kyew ell* or *See-kwell* (or "*Sequel*") but I won't judge.



“Database Languages”



You Say Tomayto, I Say Tomahto



Outline

- Introduction to Databases

- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually

- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

An Example: YouTube Videos and Comments

A few records from a relation containing information on a large number of US YouTube videos that were crawled by a researcher.

video_id	title	channel	cat_id	views
XpVt6Z1Gjjo	1 YEAR OF VLOGGING	Logan Paul Vlogs	24	4394029
cLdxuaxaQwc	My Response	Pewter	22	5845909
Ayb_2qbZHm4	Honest College Tour	CollegeHumor	23	859289
EVp4-qjWVJE	Chargers vs. Broncos	NFL	17	743947
⋮	⋮	⋮	⋮	⋮

And comments. . .

video_id	comment_id	comment	likes	replies
cLdxuaxaQwc	xl418Pq1	Love you Pooter don't apologize your fine	0	0
cLdxuaxaQwc	yV7x88ba	That word is not okay to say because. . .	0	0
Ayb_2qbZHm4	UB0bn1zz	im watching this at college	0	0
Ayb_2qbZHm4	mBb7991a	Yo this was funny af	0	0
⋮	⋮	⋮	⋮	⋮

An Example: Youtube Videos and Comments

These two relations are, well, related...

video_id	title	channel	cat_id	views
XpVt6Z1Gjjo	1 YEAR OF VLOGGING	Logan Paul Vlogs	24	4394029
cLdxuaxaQwc	My Response	Pewter	22	5845909
Ayb_2qbZHm4	Honest College Tour	CollegeHumor	23	859289
EVp4-qjWVJE	Chargers vs. Broncos	NFL	17	743947
⋮	⋮	⋮	⋮	⋮

video_id	comment_id	comment	likes	replies
cLdxuaxaQwc	xl418Pq1	Love you Pooter don't apologize your fine	0	0
cLdxuaxaQwc	yV7x88ba	That word is not okay to say because...	0	0
Ayb_2qbZHm4	UB0bn1zz	im watching this at college	0	0
Ayb_2qbZHm4	mBb7991a	Yo this was funny af	0	0
⋮	⋮	⋮	⋮	⋮

An Example: Youtube Videos and Comments

1. Video table
   - Each row is an individual video record.
   - Videos are uniquely identified by the column `video_id`.
2. Comments table
   - Each row is an individual comment record.
   - Comments are uniquely identified by the column `comment_id`.
3. Videos and comments related via the `video_id` column.

This Section

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

Primary Key

The **primary key** in a table is the **single** column—or if a single column does not exist, then it is the *simplest set of columns*—that uniquely identifies each row.

- There is **only one** primary key on a table, and **no part** of it can contain `NULL` values.
- Each table *should* have a primary key, but it is not required.
- A primary key that makes sense in a business context is called a *natural key*.
- We can instead create a new column used **only** for uniqueness, with no business meaning. This is called a *surrogate key*.
- Primary keys make data fetching more efficient.

Primary Key

Questions

1. In the `Video` table, could `video_id` and `title` be the primary key?
2. At UCLA, what would be the key for a table called `Student`?

Primary Keys: Different Perspectives

Look at the `Video` table again. There are two perspectives when looking for the primary key.

video_id	title	channel	cat_id	views
XpVt6Z1Gjjo	1 YEAR OF VLOGGING	Logan Paul Vlogs	24	4394029
cLdxuaxaQwc	My Response	Pewter	22	5845909
Ayb_2qbZHm4	Honest College Tour	CollegeHumor	23	859289
EVp4-qjWVJE	Chargers vs. Broncos	NFL	17	743947
⋮	⋮	⋮	⋮	⋮

1. In **instance**. Using only the data in the table. What is the PK?
2. In **context**. Understanding the data and what it represents. What is the PK?

In 402, you will be working in context unless told otherwise.

Primary Keys: Example

Suppose the relation R contains an automobile insurance provider's customer data. The insurance company serves all 50 states in the US. Each state maintains its own rules for license plates and driver's license numbers.

dl_number	license_plate	state	name	auto_make_model
B9614355	I♡UCLA	CA	Joe Bruin	Ford F-150
N19154	GENEBLOCK	CA	Gene Block	Tesla X
8675309	JENNY	CA	Jennifer Lopez	BMW 750i
B9614355	I♡UCLA	WA	Joe Bruin	Ford F-150
8675309	JENNY	NY	Jennifer Lopez	Tesla X
B9914561	GO RAMS	CA	Sean McVay	BMW 750i
1212121	JENNYFROMBLOCK	CA	Jennifer Lopez	AMC Pacer

What are the candidates for the primary key? Which would you pick?

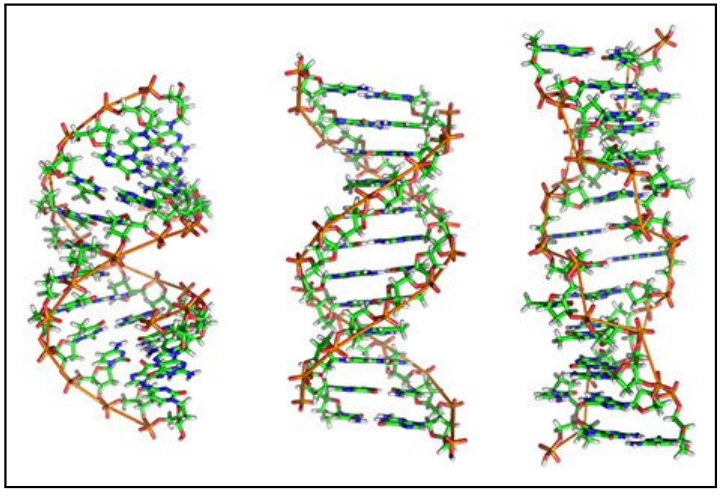
Primary Key

Exercise:

Let's think more about this.

Suppose we are the NSA and we want to create a relation containing every person that lives in the United States and data about them. **What would be the primary key?**

Primary Key



What about an index of every human on Earth?

Scientists are working diligently on decoding the human genome.

Could this be used as a primary key? Let's hope not!

This Section

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

Referential Integrity: Insertion

The column `video_id` in `Comment` *relates* each comment to a video in `Video` via its copy of `video_id`. Note that the columns need not have the same name.

Therefore, a **video must exist before we record any comments**. If we try to insert a comment with a non-existent `video_id`, we violate this rule.

This forms part of the **referential integrity** concept, which prevents insert anomalies.

Referential Integrity: Update

What happens if we change a `video_id` in the `Video` table?

If we update a `video_id` in either table, the relationship between them can break.

- If we change a `video_id` in the **referred** (parent) table, `Video`, all matching rows in `Comment` become orphans.
- If we change a `video_id` in the **referring** (child) table, `Comment`, the comment may no longer match any video.

For example, if we change a `video_id` in `Video` to something random like `foo`, the comments that pointed to the old ID will now be orphaned.

Referential Integrity: Update

Video

video_id	title	channel	views
XpVt6Z1Gjjo	1 YEAR OF VLOGGING	Logan Paul Vlogs	4394029
cLdxuaxaQwc	My Response	Pewter	5845909

Comment

video_id	comment_id	comment	likes	replies
cLdxuaxaQwc	xl418Pq1	Love you Pooter don't apologize your fine	0	0

Questions

1. Which table is referred (parent)? Which is referring (child)?
2. What if we change `video_id` in `Comment`?
3. How could we restore integrity?

Referential Integrity: Delete (Your Turn)

Questions

1. What happens if we delete a `Video`?
2. What happens if we delete a `Comment`?
3. Which case is worse, and how could we restore integrity?

Foreign Key

`video_id` in `Comment` is a **foreign key** referencing `video_id` in `Video`.

The foreign key establishes that values of a column (or set of columns) in the referring (child) table must exist in the referred (parent) table for referential integrity to be maintained.

Exercise

Video

video_id	title	channel	cat_id	views
XpVt6Z1Gjjo	1 YEAR OF VLOGGING	Logan Paul Vlogs	24	4394029
cLdxuaxaQwc	My Response	Pewter	22	5845909
Ayb_2qbZHm4	Honest College Tour	CollegeHumor	23	859289
EVp4-qjWVJE	Chargers vs. Broncos	NFL	17	743947
⋮	⋮	⋮	⋮	⋮

Comment

video_id	comment_id	comment	likes	replies
cLdxuaxaQwc	xl418Pq1	Love you Pooter don't apologize your fine	0	0
cLdxuaxaQwc	yV7x88ba	That word is not okay to say because...	0	0
Ayb_2qbZHm4	UB0bn1zz	im watching this at college	0	0
Ayb_2qbZHm4	mBb7991a	Yo this was funny af	0	0
⋮	⋮	⋮	⋮	⋮

Question: Is `video_id` in `Video` a foreign key into `video_id` of `Comment`?

Foreign Keys: The Fine Print

A few more rules govern how foreign keys connect a child table to its parent:

- Foreign keys refer to **unique keys** in the referred (parent) table, not necessarily a primary key.
- A foreign key can use more than one column. This is called a **composite foreign key**. All the columns together must match the same combination in the other table.
- If you also need a relationship using only part of those columns, you must create another foreign key for that subset.
- The way databases handle `NULL` in foreign keys **varies by system**. (Some allow it, some treat it as "no relationship.")

Composite Foreign Keys

Composite foreign keys are not the most common construct in SQL, but let's take a look. What are the FKs?

`ProductCatalog`

SupplierID	ProductID	ProductName	Price
10	1001	Three Course Meal Gum	15.99
10	1002	Mai Tai Cocktail Mix	9.49
11	2001	Hibiscus Seeds	6.99
11	1002	Everlasting Gobstoppers	8.50
12	3001	Hot Chocolate Mix	5.75

`OrderLine`

OrderLineID	OrderID	SupplierID	ProductID	Quantity	LineTotal
1	500	10	1001	2	31.98
2	500	11	2001	1	6.99
3	501	10	1002	3	28.47
4	502	12	3001	4	23.00
5	503	11	2002	2	17.00

Foreign Keys are used for Referential Integrity!

Foreign keys are used to impose **referential integrity constraints. Period. End of story.**.

You may also note though that we can **merge** both tables together to get data about video and comments in one single view.

This is a separate concept and has nothing to do with foreign keys.

Referential Integrity: `CASCADE` and `SET NULL`

If we **change or delete** a value in the **unique key or primary key** that a foreign key refers to, the database will give an error unless we tell it what to do.

- `CASCADE` $\rightarrow$ also update or delete the matching rows in the referring (child) table.
- `SET NULL` $\rightarrow$ replace the foreign key value with `NULL` and end the relationship.

This Section

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

Superkeys and Candidate Keys

There can be **many** sets of columns that uniquely identify each row.

- A **superkey** is *any* set of columns that uniquely identifies each row.
- A **candidate key** is a *minimal* superkey: drop any column and it no longer identifies each row.
- The **primary key** is the **one** candidate key we choose. The others can be enforced as **unique keys**.

In the `Video` table:

- {`video_id`} is a candidate key, and our primary key.
- {`video_id`, `title`} is a superkey, but not a candidate key.
- All five columns together form a superkey, as long as no two rows are identical.

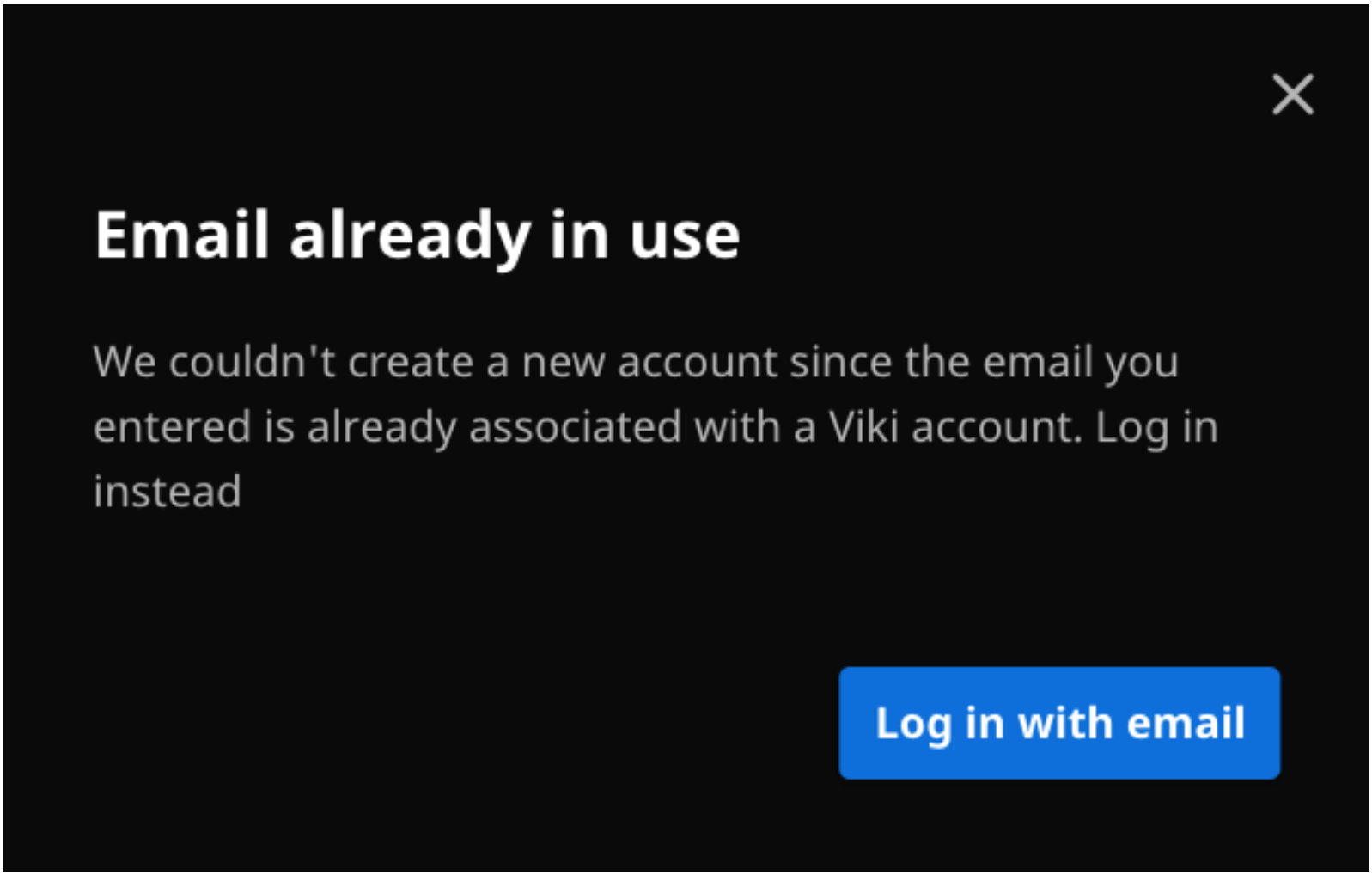
Unique Keys

Primary keys are unique and used to *identify rows in a table*. They cannot contain `NULL` values. Unique keys:

- are other keys that must be unique *but are not used to identify rows* in a table
- can have `NULL` values. `NULL` cannot be compared – they are all considered unique

A UNIQUE Example

Ever seen this message?



A UNIQUE Example

Primary key: customer_id. Also unique: email, ccnum.

```
CREATE TABLE customer (
  customer_id INTEGER PRIMARY KEY,
  name VARCHAR(255) NOT NULL,
  email VARCHAR(255) NOT NULL UNIQUE,
  ccnum VARCHAR(16) NOT NULL UNIQUE
);
```

Rows 6 and 7 would not be permitted:

customer

customer_id	name	email	ccnum
1	Sally Struthers	sally@struthers.com	1234567890123445
2	Ronald McDonald	boss@mcdonalds.com	2525252525252525
3	Joe Bruin	joebruin@ucla.edu	1111111111111111
4	Josie Bruin	josiebruin@ucla.edu	1919191919191919
5	Julio Frenk	chancellor@ucla.edu	0000000000000000
~~6~~	~~Gene Block~~	~~chancellor@ucla.edu~~	~~9876543210012345~~
~~7~~	~~Ronald McDonald Jr.~~	~~junior@mcdonads.com~~	~~2525252525252525~~

Primary Key vs. Unique Key

Feature	Primary Key (PK)	Unique Key (UK)
Purpose	Identifies each row in a table	Ensures a column or set of columns have unique values
Nulls allowed	No	Yes (treated as different from other NULLs)
Number per table	Only one	Can have many
Used by foreign keys	Commonly referenced	Can also be referenced
Automatically indexed	Yes	Yes (usually automatically)

Summary: Keys

1. The **primary key** *uniquely identifies* each row. There is exactly one, and it cannot contain `NULL` values. When there are several candidate keys, pick
   - the simplest, or
   - the one maintained by the institution.

2. A **foreign key** is a column (or set of columns) in the referring (child) table whose values must exist in the referred (parent) table. This maintains referential integrity.

3. A **unique key** is a candidate key we did not choose as the primary key. Its values must be unique, but it can contain `NULL` values.

This Section

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

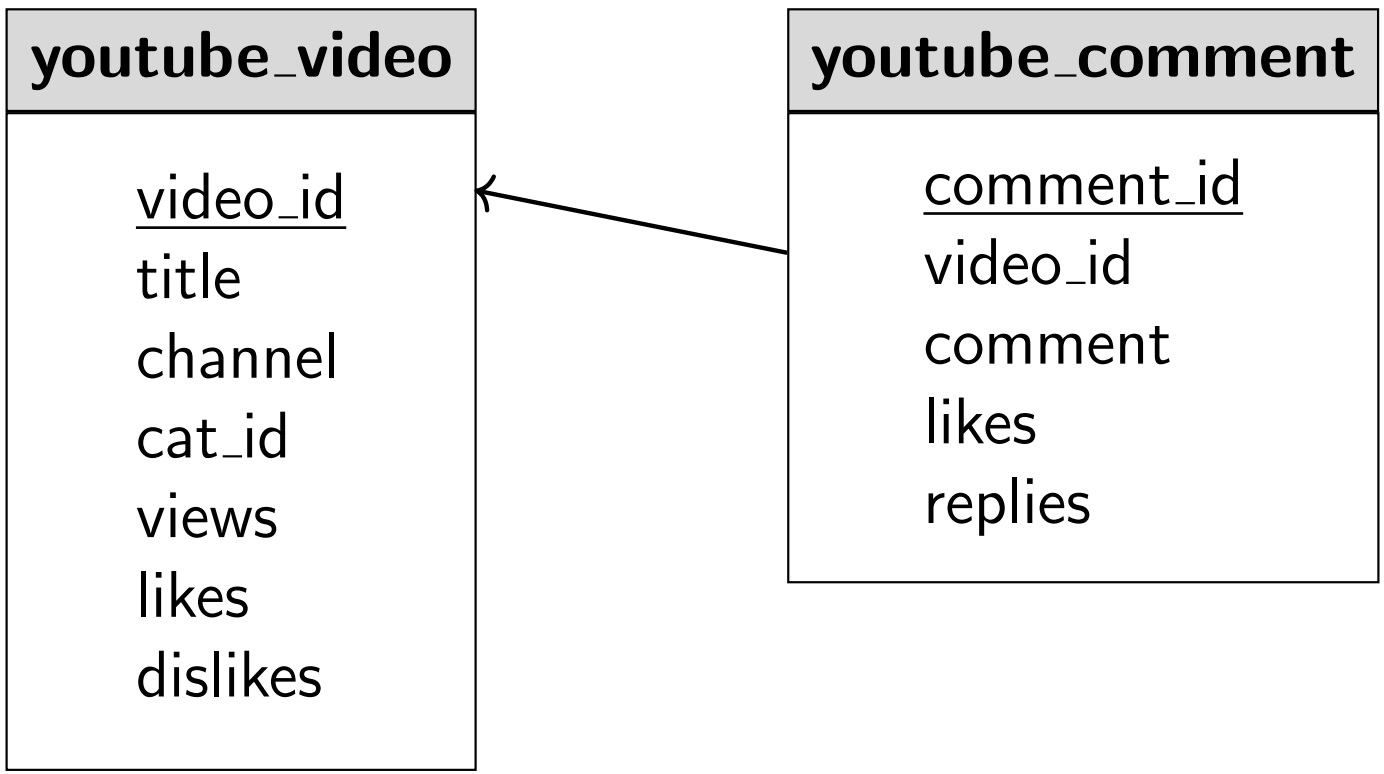
Representing a Schema Visually: Word Soup

Tables, columns/attributes and keys (both primary and foreign) are represented visually in a diagram.

- A box for each table with the name of the table at the top of the box, with the background of the name sometimes shaded and separated from the rest of the box with a horizontal line;
- Attributes/columns are listed beneath the name of each table, **primary keys underlined**, followed by all other attributes/columns.
- An arrow is drawn from each foreign key in the referring (child) table to the unique key of the referenced/referred (parent) table.

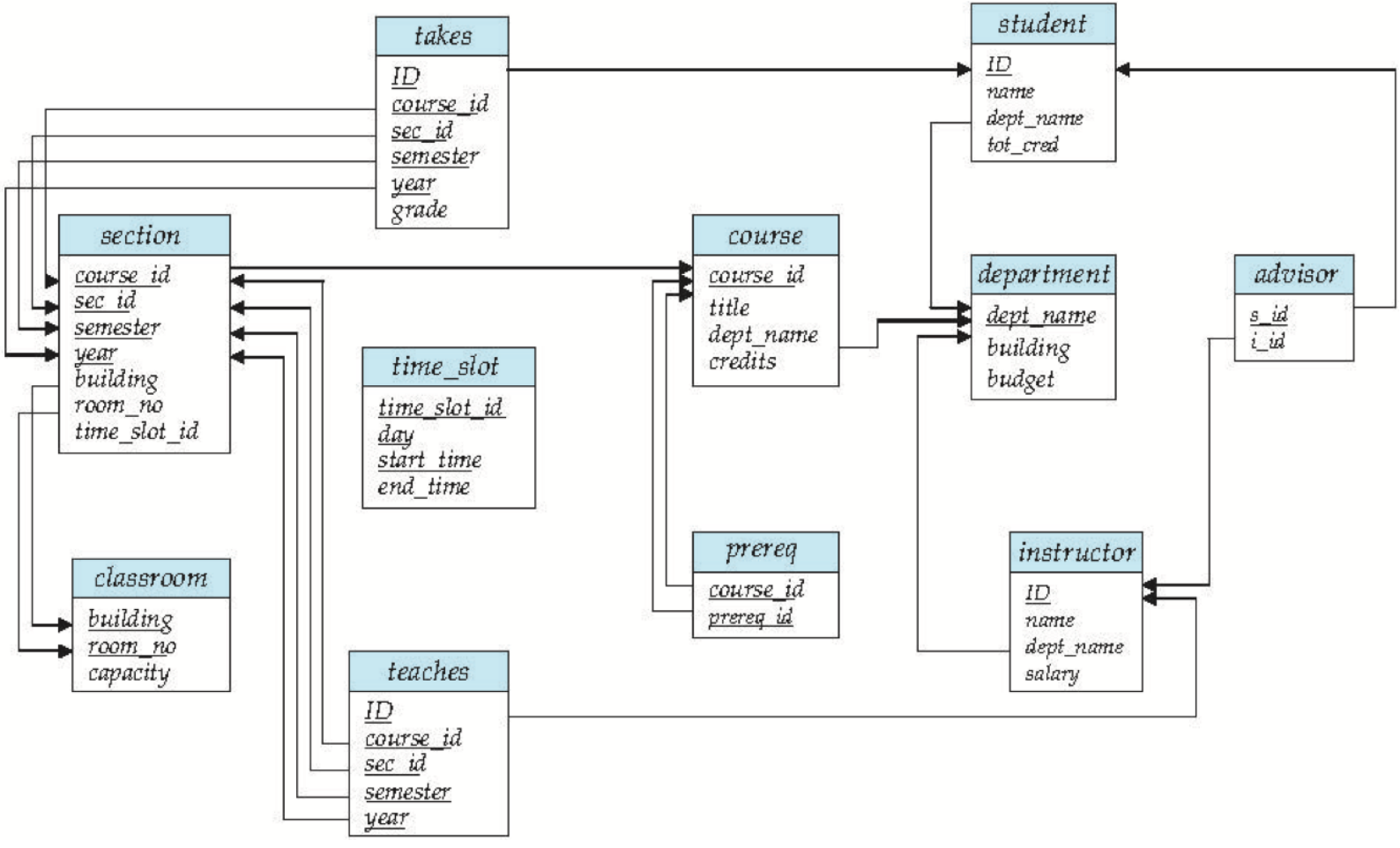
Representing a Schema Visually: An Example

For the original YouTube example, we would have the following.



Representing a Schema Visually

These visualizations can get very complicated very quickly. Here is an example from the text regarding the university schema.



Outline

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

Data Definition Language (DDL)

The DDL specifies:

1. The schema of the database and each table
2. The types of each column (attribute)
3. Integrity constraints
4. The set of indexes on the tables
5. Who is allowed to do what to the database and tables
6. How the data are stored on disk

Right now we will focus on 1 and 2.

This Section

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

ANSI SQL Data Types

Some rudimentary data types are defined by the SQL standard:

1. Numeric: `smallint`, `int(eger)`, `bigint`, `numeric(p,d)` / `decimal(p,d)`, `real`, `double precision`, `float`
2. String/Text: `char(n)`, `varchar(n)`
3. Boolean: `boolean`
4. Binary: `binary(n)`, `varbinary(n)`, `blob`, but support varies (PostgreSQL uses `bytea`)
5. Date/Time: `date`, `time`, `timestamp`, `interval`

`NULL` is not a type: it marks a missing value in a column of any type. Comparing with = gives `UNKNOWN`, so use `IS NULL`.

Integer Types (Exact)

PostgreSQL offers the standard integer type in three different sizes for our computing pleasure.

`smallint`

`integer` (also called `int`)

`bigint`

All three are signed. PostgreSQL has no unsigned integers and no `tinyint` or `mediumint`.

Integer Types (Exact)

Type	Storage	Min	Max
`SMALLINT`	2 bytes	-32768	32767
`INT(EGER)`	4 bytes	-2147483648	2147483647
`BIGINT`	8 bytes	-2^{63}	$2^{63}-1$

When picking a data type, there's an important tradeoff:

- be efficient to save space, and...
- plan for future use

We *promote* types to increase precision or size and never *demote* which may cause truncation.

Fixed-Point Types (Arbitrary Precision)

When we need **exact** fixed point precision, we can use `decimal` (a.k.a `numeric`) and is defined as

`DECIMAL(p, d)`

`p` is called the precision (total number of digits) and `d` is called the scale (digits after the decimal point).

Both names are SQL standard. In PostgreSQL, `NUMERIC` with no `(p, d)` stores values of any size exactly.

What are some situations where we would want to use this?

Floating Point Types

The SQL standard also defines floating-point types: `real`, `double precision`, and `float(p)`. PostgreSQL supports all three.

Floating point numbers are dangerous to compare as they are approximate not exact.

Type	Storage	Precision	Typical Range
`REAL`	4 bytes	≈6 digits	$\approx 10^{-37}$ to 10^{37}
`DOUBLE PRECISION`	8 bytes	≈15 digits	$\approx 10^{-307}$ to 10^{308}

In PostgreSQL, `FLOAT` with no precision means `DOUBLE PRECISION`; `FLOAT(1)` to `FLOAT(24)` mean `REAL`.

Which floating point type would we use for a spaceship? What about for a machine learning algorithm?

CHAR, VARCHAR, TEXT, BYTEA

- Both `CHAR(n)` and `VARCHAR(n)` store strings up to `n` **characters**. Both are SQL standard.
- `CHAR` always stores `n`, padding with spaces. `VARCHAR` stores up to `n`.
- `TEXT` stores strings with no limit. It is not in the standard, but PostgreSQL and most other databases support it.
- For *binary* data, PostgreSQL uses `BYTEA` instead of the standard `BINARY(n)` and `VARBINARY(n)`.

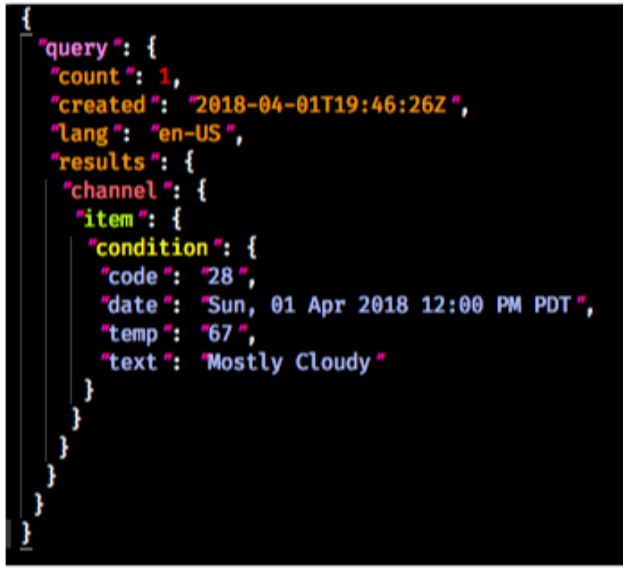
CHAR, VARCHAR, TEXT, BYTEA

Many character encodings use characters made up of multiple bytes. The count for `character` and `varchar` is *still* the number of characters.

So the letter "A" is the same size as the pile of poo emoji 💩 even though it may require multiple bytes.

Large Data

We can store larger amounts of freeform data as well. In PostgreSQL, `TEXT` and `BYTEA` each hold up to 1 GB. `JSON` and `JSONB` store JSON documents.



JSON

A `JSON` column lets us add extra data to a table without changing the schema. Prefer `JSONB`: it is faster to query and can be indexed.

Day After Day, Time After Time

The SQL standard defines five basic types for dates and times, and PostgreSQL supports all of them:

1. `DATE` for representing, well, dates.
2. `TIME` for a time of day (HH:MM:SS).
3. `TIMESTAMP` for a date and time, with **no** time zone.
4. `TIMESTAMP WITH TIME ZONE`, or `TIMESTAMPTZ` in PostgreSQL (stored in UTC; displayed in the session time zone).
5. `INTERVAL` for a duration, such as `'2 hours 30 minutes'`.

You should always use time zones (`TIMESTAMPTZ` and not `TIMESTAMP`) in your schemas, even if it is just Zulu/UTC. Use the `CURRENT_TIMESTAMP` function to get the current timestamp.

Day After Day, Time After Time

Dates and times, in my opinion, are one of the most frustrating data types to work with in a (programming) language.

You will want to refer to the documentation when working with dates and times for this class.

See here.

Multiple Choice: ENUM

- In PostgreSQL, an ENUM is a type whose values can take on **only one value from a fixed list**. It is not part of the SQL standard.
- We first create the type, then use it for a column:

```
CREATE TYPE tshirt_size AS ENUM
    ('small', 'medium', 'large', 'xlarge');

CREATE TABLE person (
    student_name VARCHAR(255),
    tshirt_size  tshirt_size
);
```

- The order of the values is defined when the type is created, so `'small' < 'large'`.
- ENUMs are best for small, fixed sets of choices (like sizes or categories).
- **What is an alternative for large sets of choices?**

From Time to Space

PostgreSQL has built-in geometric types (`point`, `line`, `polygon`, `circle`), and the PostGIS extension adds full support for geographic data. Spatial or geometric data types let us represent shapes such as points, lines, and polygons. These can describe locations, boundaries, and regions on a map. We can use them to:

1. Check whether a point is inside a region (e.g., geofencing, reverse 911)
2. Find where two regions overlap or intersect (e.g., mapping or coverage analysis)
3. Identify which line or street a point lies on (e.g., vehicle tracking)
4. Find all points within a distance or radius (e.g., restaurants within 5 miles)

More Information: PostgreSQL Documentation



The PostgreSQL documentation goes into much more detail on these data types. You will want to reference it. See here.

This Section

- Introduction to Databases
- Introduction to the Relational Model
  - Primary Keys
  - Foreign Keys and Referential Integrity
  - Unique Keys
  - Representing a Schema Visually
- Data Definition Language (DDL)
  - Column Data Types
  - SQL Schemas

Data Organization in PostgreSQL

Data in PostgreSQL is organized into **tables**, tables are grouped into **schemas**, and schemas are grouped into **databases**. Several databases can exist on the same **database server**.

Each **schema** is usually designed for a different purpose. For example, in a university database, we might use separate schemas for:

- students
- faculty
- courses
- finance

In PostgreSQL, you can join tables across schemas in the same database, but not across databases.

SQL Naming Conventions for Capitalization

Common SQL style conventions:

- **SQL keywords** are written in `UPPERCASE`: `SELECT`, `FROM`, `WHERE`, `JOIN`, `GROUP BY`
- **Table and column names** are written in lowercase `snake_case`: `student`, `student_id`, `course_name`
- **Constants or string values** are written in quotes: `'Los Angeles'`, `'Finance'`, `'A'`

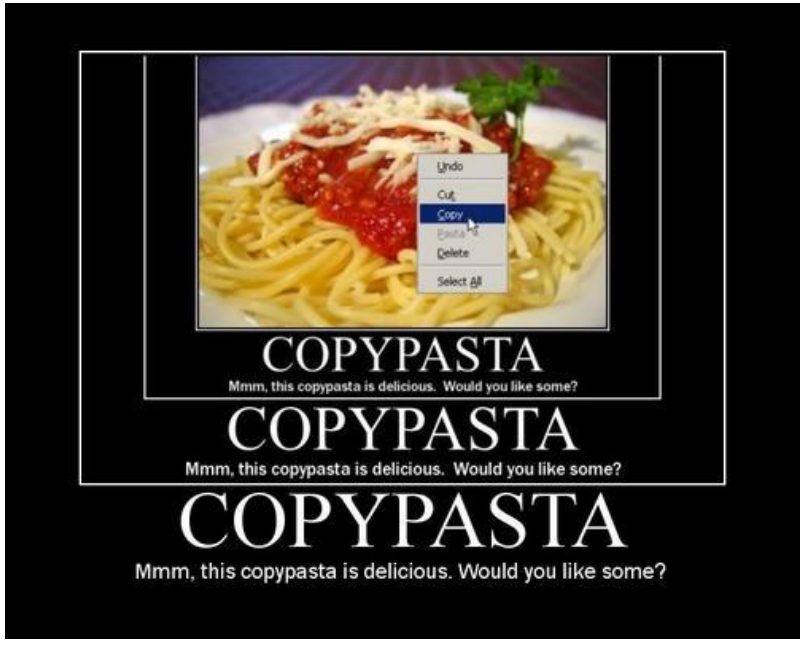
SQL is not case-sensitive for keywords, but using a consistent style makes your code easier to read.

PostgreSQL converts unquoted names to lowercase, so `courseName` becomes `coursename`. Stick to `snake_case`.

Copy and Paste Issues

Copying SQL code directly from slides or websites often causes errors due to hidden formatting, quotes, or comments. To avoid problems:

- Paste into a plain text editor first and fix quotation marks.
- Check that comment dashes are two plain hyphens (`--`), not an en or em dash, before running in `psql`.
- Use straight quotes (`' '`) for string literals.



Creating a Table

We create a table using the `CREATE TABLE` syntax. Let's give the `youtube_video` table, a facelift and add some more attributes:

Name	Type	NOT NULL?	DEFAULT?	PK?	FK?
post_date					
duration					
description					
privacy					
content					
extra_data					

Creating a Table

```
CREATE TYPE privacy_level AS ENUM ('public', 'private', 'unlisted');

CREATE TABLE youtube_video (
  video_id      CHAR(11) PRIMARY KEY,
  title         VARCHAR(100) NOT NULL,
  channel       CHAR(24) NOT NULL,
  cat_id        SMALLINT DEFAULT 0,
    -- fewer than 50, so could also be an ENUM type
  likes         INT DEFAULT 0,
    -- can always promote
  dislikes      INT DEFAULT 0,
  views         BIGINT DEFAULT 0,
    -- popular videos exceed INT's ~2.1 billion
  post_date     TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP,
  duration      INTERVAL DEFAULT '0 seconds',
    -- '2h 30m 5s' -> 02:30:05
  description   TEXT,
  privacy       privacy_level NOT NULL DEFAULT 'public',
  content       BYTEA NOT NULL,
  extra_data    JSONB,
  FOREIGN KEY (channel) REFERENCES youtube_channel(channel_id)
);
```

Creating a Table

The general pattern for defining a column in SQL is:

```
column_name column_data_type [NOT NULL]
    [DEFAULT value_or_function]
```

Optional parts (in brackets) specify constraints or default values.

Example. Let's look at an example with `bruinbase`.

Creating a Table

Some things to note:

- as you can probably tell, two dashes denote a comment.
- we specify that a particular column must not be null by adding `NOT NULL` after the data type
- recall that the primary key must be unique and must be not null
- we can also add default values for a column by specifying the keyword `DEFAULT` followed by a particular value (e.g. `CURRENT_TIMESTAMP` for a timestamp)
- primary and foreign keys can be specified **inline** if they consist of a single column, but must be specified at the end otherwise.

Changing a Table's Schema

Great care must be taken when we design a database schema or a table schema. We do not want to take the decision to modify the schema lightly.

1. **Need larger numbers or strings?** Promote types.
2. **Need to delete a column?** Just stop using it.
3. **Need to add a column?** This one is trickier:
   - Create a separate table with the column and merge.
   - Create an extra column (JSON), a priori, in the table as a "catch-all"

Best practice. Usually, when a table is created, it's set in soft stone except for promoting data types.

Changing a Table's Schema

```
ALTER TABLE youtube_video
  DROP COLUMN cat_id,
  ALTER COLUMN video_id TYPE CHAR(12),
      /* widen to 12 */
  ADD COLUMN flagged BOOLEAN DEFAULT FALSE,
  ADD COLUMN rowno INT,
  DROP CONSTRAINT youtube_video_pkey;

ALTER TABLE youtube_video RENAME TO youtube_videos;
```